

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: AI-Based Resume Screening & Candidate Shortlisting System

SUBMITTED BY:

Rashmi Bhagat (Roll No: 2400321540155)

Degree: Bachelor of Technology (B.Tech)

Branch: Computer Science & Engineering (Data Science)

Academic Year: 2026 - 2027

UNDER THE GUIDANCE OF:

Mr. Aditya

Project Mentor / Guide

Department of Computer Science & Engg.

Status: 3rd Year Project Specification

Date of Submission: 30-10-2026

Document Revision & Approval History

0.1	01-10-2026	Initial Draft & Scope Definition	Rashmi Bhagat	Mr. Aditya
-----	------------	----------------------------------	---------------	------------

Guidance Note: This Software Requirements Specification document has been prepared specifically for the AI-Based Resume Screening & Candidate Shortlisting System. It conforms to the IEEE 830-1998 standard recommended practice and is strictly designed for 3rd-year university lab evaluation, project reviews, and system design baselines. All requirements, constraints, and design models documented herein are scoped exclusively to the ML-driven resume parsing, skill extraction, candidate scoring, and shortlisting workflow of this project.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance targets, and constraints of the proposed system: **'AI-Based Resume Screening & Candidate Shortlisting System'**. This specification serves as a binding reference agreement between the student developer, project evaluators, academic supervisor, and prospective end users during the 3rd-year engineering curriculum.

1.2 Scope of the System

The proposed application is designed to solve the inefficiency and inherent subjectivity of manual resume screening during high-volume recruitment drives by delivering an end-to-end automated platform. Key boundary guidelines include:

- In-Scope Capabilities:** Centralized ingestion of resumes (PDF/DOCX) and job postings; rule-based NLP skill extraction with vocabulary matching and alias resolution; TF-IDF cosine similarity between resume and job description; machine-learning-based resume scoring (0–100 scale); automated shortlisting decisions (Shortlisted / Review / Rejected); an interactive recruitment analytics dashboard with KPI cards and charts; relational database persistence with 7 normalized tables; and cloud deployment with automatic SQLite fallback and model bootstrap.
- Out-of-Scope Boundaries:** Live video interview processing and real-time candidate verification; web scraping from LinkedIn, Naukri, or other job portals; enterprise ATS (Applicant Tracking System) integrations; commercial payment gateway or subscription billing; and multi-language resume support (current scope: English only).
- Expected Benefits:** Reduction of screening turnaround time by approximately 60%, reduced subjectivity in initial-stage shortlisting, consistent scoring criteria applied uniformly across all candidates, and immediate analytics availability across devices (desktop to mobile).

1.3 Definitions, Acronyms, and Abbreviations

Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Architecture	API	Application Programming Interface for data interchange via JSON/HTTPS.
Security	RBAC	Role-Based Access Control for admin/HR/recruiter roles.
Database	CRUD	Create, Read, Update, Delete basic data manipulation operations.
Database	ORM	Object-Relational Mapping (SQLAlchemy used in this project).
Network	REST	Representational State Transfer stateless software architectural style.
ML	ML	Machine Learning.
ML	NLP	Natural Language Processing.
ML	TF-IDF	Term Frequency-Inverse Document Frequency (used for JD similarity).
ML	GradientBoosting	Ensemble regression model (final selected model).
ML	RandomForest	Ensemble baseline model (compared, not selected).
ML	MAE	Mean Absolute Error (evaluation metric).
ML	RMSE	Root Mean Squared Error (evaluation metric).
ML	R²	Coefficient of Determination (evaluation metric).
ML	Feature Engineering	Transformation of raw text into 29 numerical features.
Document	PyMuPDF / fitz	PDF parsing library for text extraction.
Document	python-docx	DOCX parsing library for paragraphs and tables.
UI	Streamlit	Python web application framework used for the UI.
UI	Plotly	Interactive charting and visualization library.
Project Specific	Resume Score	Predicted 0–100 score for candidate-job fit.
Project Specific	Skill Match %	Percentage overlap between resume skills and required skills.
Project Specific	JD Similarity	TF-IDF cosine similarity between resume and job description.

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.
- **Scikit-learn Documentation:** <https://scikit-learn.org>
- **PyMuPDF Documentation:** <https://pymupdf.readthedocs.io>
- **Streamlit Documentation:** <https://docs.streamlit.io>
- **SQLAlchemy Documentation:** <https://docs.sqlalchemy.org>
- **Plotly Documentation:** <https://plotly.com/python>
- **Passlib & bcrypt Documentation:** <https://passlib.readthedocs.io>
- **GitHub Repository:** https://github.com/theamri24/HCL_Ai-Based-Resume-Screening-and-Candidate-Shortlisting-System-dataset

1.5 Document Overview

The remainder of this SRS is partitioned into structural sections: Section 2 outlines the product perspective, target user personas, operational context, and design constraints. Section 3 itemizes detailed functional requirements organized into logical software modules. Section 4 specifies external interfaces (UI, Hardware, Software, Network). Section 5 defines non-functional attributes (Performance, Reliability, Security). Finally, Section 6 provides standard academic modeling diagrams.

2. Overall Description

2.1 Product Perspective

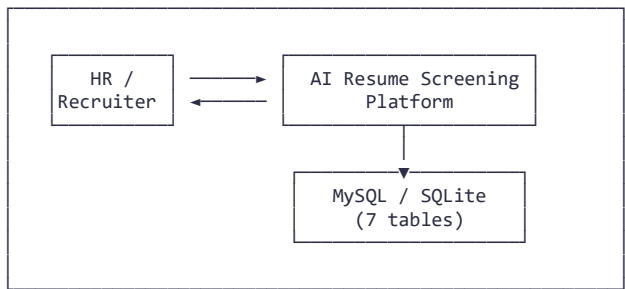
The AI-Based Resume Screening & Candidate Shortlisting System is structured as a self-contained multi-tier system comprising a presentation layer, an application/logic tier, and a persistent database backend:

- **Presentation Layer** — Streamlit-based web UI (login, dashboard, jobs management, resume upload, candidates listing, analytics).
- **Application/Logic Tier** — Python service layer with 7 modules: resume parser, NLP engine, feature engineering, ML prediction, database ORM, model bootstrap, and seed data.
- **Persistence Layer** — MySQL 8.x (local development) / SQLite 3 (cloud fallback with auto-detection).

The system interfaces with document parsing libraries (PyMuPDF, python-docx), ML libraries (scikit-learn, joblib), NLP utilities (TF-IDF vectorizer, regex), and UI libraries (Streamlit, Plotly) through internal Python imports. No external REST API is exposed in the current scope.

System Context Diagram (Level 0)

High-level system boundary showing external entities interacting with the central application.



External Entities:

- HR/Recruiter (primary user: creates jobs, uploads resumes)
- Administrator (maintains models, DB, deployment)
- File System (uploads folder for PDF/DOCX storage)
- ML Model Artifacts (model.pkl, encoders.pkl, feature_columns.pkl)

2.2 User Classes and Characteristics

The target software accommodates distinct user personas with varying access hierarchies:

System Administrator	High (DevOps / DB Management)	Complete access: user role assignment, model retraining, database backup/restore, system configuration.
HR / Recruiter	Intermediate (General Web User)	Create job postings, upload candidate resumes (PDF/DOCX), view scores and decisions, filter candidates, view analytics dashboard.
Guest / Public Visitor	Basic	Read-only access to the login page and system overview. No data access without valid credentials.

2.3 Operating Environment

The application operates within the following software and hardware environments:

- **Client Requirements:** Modern web browser (Google Chrome >= v110, Mozilla Firefox >= v108, Microsoft Edge, Safari); minimum display resolution 1366x768 for desktop, responsive view down to 360x640 for mobile.
- **Server Runtime Environment:** Local development on Windows 10/11 with Python 3.10+; cloud deployment on Linux (Ubuntu 22.04 LTS) via Streamlit Community Cloud; development IDE: Visual Studio Code.

- **Database Storage Tier:** Local: MySQL 8.0 (MySQL Workbench, TCP port 3306); Cloud: SQLite 3 (file-based, auto-created on first run).

2.4 Design and Implementation Constraints

- **Academic Budget Constraint:** Utilization of free-tier cloud resources (Streamlit Community Cloud, GitHub public repository).
- **Security Compliance:** Passwords stored exclusively as bcrypt hashes (12 rounds with per-user salt); database credentials isolated in the .env file (never committed to Git).
- **Modularity Standards:** Adherence to a modular service pattern: a services/ layer with clear separation of concerns (resume parser, NLP, feature engineering, prediction, database).
- **Model Artifact Constraint:** .pkl files are binary artifacts. The system auto-detects missing model files and triggers retraining from the dataset on first cloud run.
- **Database Compatibility Constraint:** SQL must run on both MySQL (local) and SQLite (cloud) without modification; dialect-specific commands are abstracted in the service layer.

2.5 Assumptions and Dependencies

- **Assumptions:** Users possess continuous Internet connectivity with minimal bandwidth of 2 Mbps; uploaded resumes are text-based PDF or DOCX files (not scanned images requiring OCR); candidate email addresses are unique per candidate record (used as natural key); job description text is provided in English; and historical resume data for training is available as AI_Resume_Screening.csv.
- **Dependencies:** Continuous availability of the Python Package Index (PyPI) for library installation; the training dataset (AI_Resume_Screening.csv) must be present for first-run model bootstrap on cloud; the MySQL service must be running on localhost:3306 for local development; and Streamlit Cloud must have access to the GitHub repository for auto-deployment.

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x):

3.1 Module 1: Authentication and Access Governance

FR-1.1	User Account Auto-Bootstrap	Default admin user admin@resume.com created on first run if not present.	High
FR-1.2	Secure Credential Login	Registered Email + Plain Password; verifies bcrypt hash with 12 rounds.	High
FR-1.3	Session State Management	Streamlit session_state with logged_in flag and user dict maintained across pages.	High
FR-1.4	Logout	Clears session state and returns user to login screen.	High

FR-1.5	Password Reset (Admin)	Admin can trigger bcrypt hash regeneration for existing users.	Medium
--------	------------------------	--	--------

3.2 Module 2: Job Management

FR-2.1	Job Creation	Title (required), Description (required), Required Skills (comma-separated, at least 1), Minimum Experience (0–30 years).	High
FR-2.2	Job Listing	Display all jobs with title, min experience, required skills, applicant count, and average candidate score.	High
FR-2.3	Job Detail View	Expander showing full job description.	Medium
FR-2.4	Job-User Association	Each job linked to the creating user via created_by foreign key.	Medium

3.3 Module 3: Resume Ingestion & Parsing

FR-3.1	File Upload	Accept PDF and DOCX files <=10 MB. Reject other formats with clear error.	High
FR-3.2	File Validation	Extension whitelist check (.pdf, .docx) + size limit check.	High
FR-3.3	Secure File Storage	Save with UUID-suffixed secure filename in uploads/ directory.	High
FR-3.4	PDF Text Extraction	Extract text from all pages using PyMuPDF (fitz).	High
FR-3.5	DOCX Text Extraction	Extract paragraphs + table content using python-docx.	High
FR-3.6	Text Cleaning	Normalize whitespace, remove zero-width characters, collapse multiple spaces.	High

3.4 Module 4: NLP-Based Information Extraction

FR-4.1	Skill Extraction	Rule-based matching against 24+ pre-defined skill vocabulary with word-boundary regex.	High
FR-4.2	Alias Resolution	Map variations (e.g., sklearn → scikit-learn, nodejs → node.js).	High

FR-4.3	Experience Extraction	Regex-based parsing of years-of-experience from resume text.	High
FR-4.4	Email Extraction	Regex pattern matching for candidate contact email.	Medium
FR-4.5	Phone Extraction	Regex pattern matching for candidate phone number.	Medium
FR-4.6	Keyword Extraction	Frequency-based top-20 keyword identification using Counter.	Medium
FR-4.7	Text Normalization	Lowercase conversion, punctuation preservation for skills like C++, C#.	High

3.5 Module 5: Feature Engineering & Similarity Computation

FR-5.1	Skill Match Percentage	Compute overlap between resume skills and job's required skills.	High
FR-5.2	TF-IDF Vectorization	Convert resume and job description to TF-IDF vectors (with bigrams).	High
FR-5.3	Cosine Similarity	Compute similarity between resume and JD vectors (0–100 scale).	High
FR-5.4	29-Feature Vector Construction	Build numeric + encoded + binary + category-count features in fixed order.	High
FR-5.5	Matched/Missing Skills List	Return lists of matched and missing required skills.	High

3.6 Module 6: Machine Learning Prediction

FR-6.1	Model Loading	Load model.pkl, encoders.pkl, feature_columns.pkl with caching.	High
FR-6.2	Feature Vector Assembly	Assemble input vector in exact training order using feature_columns.pkl.	High
FR-6.3	Resume Score Prediction	Predict score (0–100) using GradientBoostingRegressor.	High
FR-6.4	Score Clipping	Clip predictions to valid 0–100 range.	High
FR-6.5	Decision Generation	Apply thresholds: >=75 Shortlisted, 50–74 Review, <50 Rejected.	High
FR-6.6	Auto-Bootstrap Model	Retrain model from dataset on first cloud run if model.pkl missing.	Medium

3.7 Module 7: Database Persistence

FR-7.1	Candidate Upsert	Insert new candidate or update existing by email (natural key).	High
FR-7.2	Resume Record Storage	Store file metadata + full extracted text (LONGTEXT).	High
FR-7.3	Prediction Record Storage	Store score, skill match, similarity, matched/missing skills, decision.	High
FR-7.4	Candidate-Skill Mapping	Insert many-to-many records in candidate_skills table (dedup with UNIQUE constraint).	High
FR-7.5	Environment Auto-Detection	Use MySQL if .env present; else SQLite with auto-created schema.	High

3.8 Module 8: Reporting & Visualization

FR-8.1	KPI Dashboard	Display total candidates, shortlisted, review, rejected, average score.	High
FR-8.2	Decision Breakdown Chart	Horizontal bars showing count and share per decision.	High
FR-8.3	Score Distribution Chart	Candidates grouped by score bands (Excellent/Good/Fair/Poor).	High
FR-8.4	Recent Activity Feed	Latest 6 screenings with name, job, score, decision badge.	Medium
FR-8.5	Top Candidates List	Top 5 candidates by score with skill match percentage.	Medium
FR-8.6	Common Skills Panel	HTML bar visualization of top 10 skills across candidates.	Medium
FR-8.7	Candidate Search	Text search by name or email.	Medium
FR-8.8	Candidate Filters	Filter by job, decision, score range, and sort options.	Medium
FR-8.9	Job-wise Analytics	Candidates count and average score grouped by job.	Medium
FR-8.10	Experience vs Score Scatter	Plotly scatter with color-coded decision.	Low

4. External Interface Requirements

4.1 User Interfaces (UI)

The interface follows standard UX usability best practices (clean sidebar navigation, badge-based decision indicators, immediate validation feedback). All forms provide immediate visual feedback (success banners or inline error alerts under erroneous fields). The screens implemented are:

- **Authentication Screen:** Modern card layout with gradient brand mark and a muted demo credential hint.
- **Dashboard:** KPI cards (accent-bordered), decision breakdown chart, score bucket chart, recent activity feed, top candidates list, and common skills panel.
- **Jobs Page:** Tab 1 lists all jobs with applicant count; Tab 2 provides the create-new-job form.
- **Upload Resume Page:** Candidate details form, file uploader, and result cards (score, decision, skill match, JD similarity).
- **Candidates Page:** Filter row (search, job, decision, sort, score slider) and candidate cards with expandable details.
- **Analytics Page:** 8 KPI cards, score distribution, top skills bar chart, job-wise analytics, experience vs score scatter, and recent predictions table.

[User Interface Wireframe Mockups]

Insert screen captures of the implemented application here: (1) Authentication Screen, (2) Dashboard (User Action Workspace), (3) Analytics Console.

4.2 Hardware Interfaces

Specification of physical interaction points:

- **Standard Deployment:** Standard input/output peripherals (Keyboard, Mouse, Display $\geq 720p$).
- **Embedded / IoT Variants:** Not applicable; this is a pure software project with no hardware interfacing requirements.

4.3 Software Interfaces

- **Relational Database Connector:** SQLAlchemy ORM 2.0 interfacing with MySQL 8.0 via PyMySQL driver (TCP port 3306) or SQLite 3 (file-based).
- **Document Parsers:** PyMuPDF (fitz) 1.24 for PDF, python-docx 1.1 for DOCX.
- **ML Libraries:** scikit-learn 1.5 (GradientBoostingRegressor, TfidfVectorizer, LabelEncoder), joblib 1.4 for serialization.
- **NLP Utilities:** Python re module for regex, collections.Counter for frequency analysis.
- **UI Libraries:** Streamlit 1.36, Plotly 5.22 (with graph_objects for custom charts).
- **Security Libraries:** passlib 1.7 + bcrypt 4.0.1 for password hashing.
- **Configuration:** python-dotenv 1.0 for environment variable loading.

4.4 Communication Interfaces

- **Web Traffic Protocol:** HTTPS (via Streamlit Cloud) for cloud deployment; HTTP (localhost) for local development.

- **Data Serialization:** UTF-8 encoded, JSON-compatible data structures exchanged internally between the Streamlit UI and the Python service layer.
- **File Upload:** Multipart form data via Streamlit's file_uploader widget.
- **Database Protocol:** MySQL protocol over TCP/IP on port 3306 (local); direct file I/O for SQLite (cloud).

5. Non-Functional Requirements (NFRs)

5.1 Performance	Page load latency < 2.0s; ML inference < 500ms; model loading cached via <code>@st.cache_resource</code> (800ms to 5ms). Dashboard queries cached with <code>@st.cache_data(ttl=30)</code> .	Manual timing via <code>time.perf_counter()</code> in scripts.
5.2 Security	Zero plain-text passwords; bcrypt hashing (12 rounds with per-user salt); parameterized SQL queries to neutralize SQL injection; file upload validation (extension + size).	Manual code review + test upload of .txt file (should reject).
5.3 Reliability	System uptime $\geq 98\%$ during academic review; auto-reconnect DB pool (<code>pool_pre_ping=True</code>); auto-bootstrap model if missing.	Stress test with 50 concurrent resume uploads.
5.4 Portability	Cross-browser compatibility (Chrome, Edge, Firefox, Safari); responsive viewport (mobile to desktop); local to cloud DB auto-detection via <code>.env</code> presence.	Manual test on Chrome, Firefox, Edge.
5.5 Usability	Clean corporate UI (no emojis); Inter font; consistent navy/slate color palette; immediate feedback on forms.	User acceptance test with 5 volunteers.
5.6 Maintainability	Modular service architecture (7 service files, 5 page files); centralized constants; comprehensive docstrings.	Code review by project guide Mr. Aaditya.
5.7 Scalability	Architecture supports additional ML models and datasets without major rework; caching layers reduce DB load.	Add 10 new jobs and 100 candidates and observe performance.
5.8 Accuracy	Model achieves $R^2 \geq 0.85$, $MAE \leq 5.0$ on test set.	80/20 train-test split, cross-validation.

6. System Design and Analysis Models

University evaluation rubrics require standard graphical analysis models illustrating the dynamic and static behavior of the application:

Use Case Diagram	Maps user actors (Admin, HR, Guest) to discrete functional interactions.	UML Standard diagram + Use Case specification tables.
Data Flow Diagram (DFD)	Illustrates information flow: Level 0 (Context) and Level 1 (Modular decomposition).	Gane-Sarson / Yourdon & Coad notation flow chart.
Entity-Relationship (ERD)	Details entities, primary/foreign keys, attributes, and relationships.	Crow's Foot notation schema with 3NF normalization notes.
Activity / Sequence Diagram	Chronological sequence of messages between UI, Controller, and DB for core feature (Upload Resume).	UML Sequence diagram detailing happy-path and error recovery.

6.1 Use Case Diagram (Text Representation)

Primary Actor: HR / Recruiter

- Login to System
- Create Job Posting
- View All Jobs
- Upload Candidate Resume
- Analyze Resume
- View Candidate Score & Decision
- Search Candidates
- Filter Candidates by Decision/Score
- View Analytics Dashboard
- Logout

Secondary Actor: Administrator

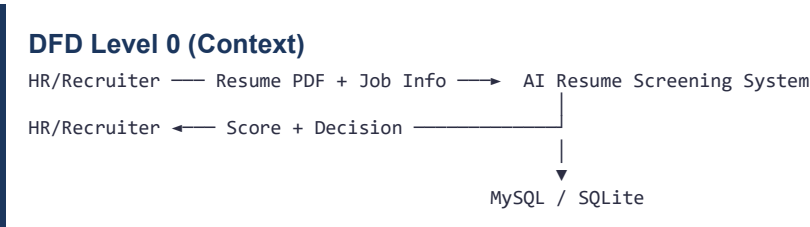
- Maintain Model Configuration
- Trigger Model Retraining
- Manage Database (backup/restore)
- Deploy System Updates

Guest Actor:

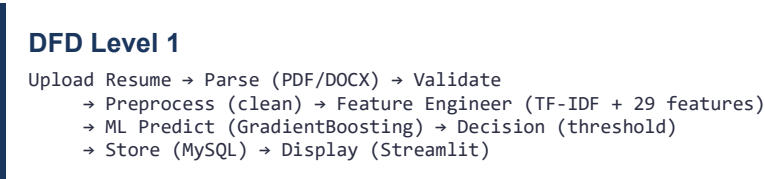
- View Login Page

6.2 Data Flow Diagram (Text Representation)

Context Level (Level 0):



Level 1 (Modular Decomposition):

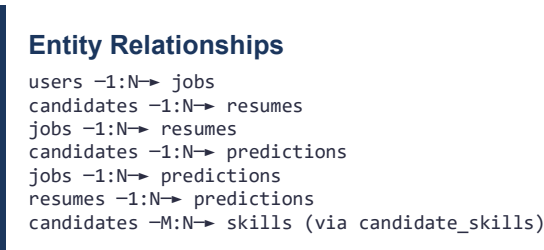


6.3 Entity-Relationship Diagram (Text Representation)

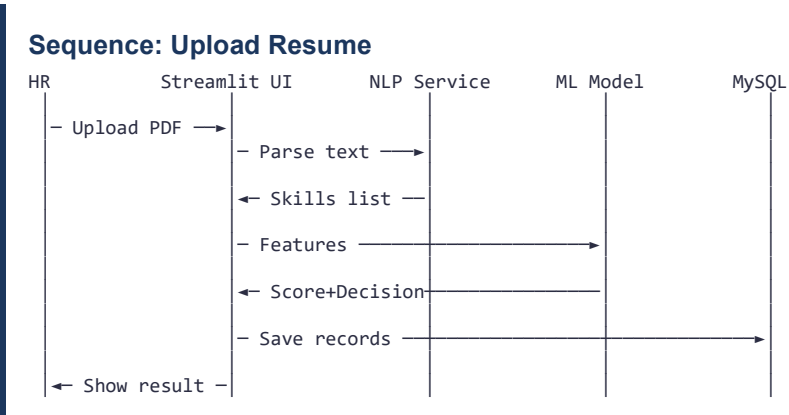
Entities:

- **users** (id, name, email, password_hash, role, is_active)
- **jobs** (id, title, description, required_skills, minimum_experience, created_by)
- **candidates** (id, name, email, phone, education, experience)
- **resumes** (id, candidate_id, job_id, original_filename, file_path, file_type, extracted_text)
- **skills** (id, skill_name, category)
- **candidate_skills** (id, candidate_id, skill_id, resume_id)
- **predictions** (id, resume_id, job_id, candidate_id, resume_score, skill_match_percentage, similarity_score, matched_skills, missing_skills, decision, model_version)

Relationships:



6.4 Sequence Diagram (Upload Resume Flow)



7. Academic Review & Sign-Off

This Software Requirements Specification document has been submitted and reviewed by the academic supervisory panel:

Name: Mr. Aaditya
Signature: _____
Date: _____

Name: _____
Signature: _____
Date: _____

Name: _____
Signature: _____
Date: _____